Insurance Application Using RAG

1. Title Page

Project Title: Insurance Application Using RAG

Prepared By: Student / Developer

Organization/College: Insurance RAG Project

Date: 06 August 2026

2. Abstract

This project presents an intelligent insurance application that uses Retrieval-Augmented Generation (RAG) to help users understand insurance policies, upload important documents, and receive grounded answers about claim eligibility and policy coverage. Traditional insurance support systems often depend on manual inspection of long documents, which is slow and error-prone. This project addresses that issue by combining document processing, embeddings, vector search, and large language models to create an AI-powered insurance assistant.

The system allows users to upload insurance policies, medical reports, hospital bills, and prescriptions. The uploaded content is processed, converted into embeddings, stored in a vector database, and later retrieved to answer user questions with evidence-based context. The expected outcome is faster policy understanding, better claim support, and reduced manual effort for insurance teams.

3. Introduction

Insurance is a financial service that provides protection against unexpected events such as accidents, health issues, or loss of property. In everyday operations, insurance companies deal with large volumes of documents, including policies, claim forms, medical reports, and supporting receipts. Processing these documents manually is time-consuming and often leads to delays.

Traditional insurance claim processing faces several challenges:

- Large and complex documents

- Difficulty in locating relevant policy clauses

- Human errors during verification

- Slow response to customer queries

- Limited availability of support staff

Artificial Intelligence is useful in this domain because it can automate document understanding and provide quick responses. However, a standalone LLM may generate answers without knowing the actual policy content. RAG is preferred because it retrieves relevant information from uploaded documents before generating an answer, making the response more accurate, grounded, and explainable.

4. Problem Statement

The insurance industry often relies on manual review of lengthy documents. Users frequently struggle to understand policy coverage, exclusions, and claim requirements. Claim verification requires reviewing multiple documents such as policies, medical reports, and bills. These tasks consume significant time and can result in delays or inconsistent decisions.

This project addresses the following problems:

- Manual verification is time-consuming.

- Policy documents are lengthy and complex.

- Users struggle to understand coverage and terms.

- Claim verification requires checking multiple supporting documents.

5. Objectives

The main objectives of this project are:

- Develop an AI-powered insurance assistant.

- Allow users to upload insurance-related documents.

- Answer policy-related questions using RAG.

- Support claim eligibility analysis.

- Reduce claim processing time and improve user experience.

6. Scope of the Project

The project focuses on the following scope:

- Supported insurance types: health, vehicle, and general policy documents

- Supported document formats: PDF, image-based documents, and text-based files

- Features included: document upload, indexing, chatbot queries, claim assistance, and dashboard monitoring

- Future enhancements: multilingual support, voice interaction, and integration with external insurance systems

7. System Architecture

The system is designed as a web-based application where users interact with a frontend, which sends requests to a backend service. The backend processes uploaded documents, extracts text, splits it into chunks, generates embeddings, stores them in ChromaDB, and uses a retrieval mechanism to answer

questions using an LLM.

Architecture diagram:

```text

User

■

Frontend (React)

■

FastAPI Backend

■

■■■■■■■■■■■■■■■■■■■

■ Document ■

■ Processing ■

■■■■■■■■■■■■■■■■■■■

■

Text Splitting

■

Embeddings

■

ChromaDB

■

Retriever

■

LLM

■

Answer

```

The current project implementation includes:

- React frontend for user interaction

- FastAPI backend for APIs and business logic

- ChromaDB for vector storage

- Gemini embeddings for semantic search
- Retrieval pipeline for grounded answers

8. Technology Stack

Layer	Technology
Frontend	React, Vite
Backend	FastAPI
Language	Python
Vector Database	ChromaDB
Database	SQLite / PostgreSQL-ready design
Embedding Model	Gemini Embeddings
LLM	Gemini / LLM-based generation
Framework	LangChain-compatible flow
Version Control	Git

9. Functional Requirements

The system provides the following functional requirements:

- User login and registration
- Upload insurance policy documents
- Upload medical reports and bills
- View uploaded documents
- Ask insurance-related questions
- Generate AI-based claim insights
- Show claim summaries and results
- Access dashboard and analytics information

10. Non-Functional Requirements

The system is designed with the following non-functional requirements:

- Security: authentication and role-based access
- Performance: fast retrieval for user queries
- Scalability: support for additional documents and users
- Reliability: stable backend and retrieval flow
- Availability: services should remain accessible during normal usage
- Maintainability: modular backend and frontend structure

11. Modules

Authentication Module

This module handles user registration, login, and role-based access.

Document Upload Module

This module allows users to upload policy documents, reports, and bills.

Document Processing Module

This module extracts text and prepares documents for indexing.

Embedding Generation Module

This module converts document chunks into vector embeddings for semantic search.

ChromaDB Storage Module

This module stores document embeddings and metadata in ChromaDB for retrieval.

RAG Retrieval Module

This module searches the vector database for relevant document chunks based on the user query.

AI Response Generation Module

This module uses the retrieved context and a language model to generate grounded responses.

Claim Verification Module

This module supports claim-related analysis using retrieved policy and evidence context.

Dashboard Module

This module provides insights about uploaded documents, claims, and AI activity.

12. Workflow

The workflow of the system is as follows:

```text
User Uploads PDF
↓
Extract Text
↓
Split into Chunks
↓
Generate Embeddings
↓
Store in ChromaDB
↓
User asks Question
↓
Retriever fetches Similar Chunks
↓
LLM generates Response
↓
Return Answer
```

13. Database Design

The project uses both relational storage and vector storage.

Relational Database

The relational database stores application data such as:

- Users
- Policies
- Claims
- Uploaded Documents

- Claim History

Vector Database

ChromaDB stores:

- Document chunks

- Embeddings

- Metadata such as source, page, and chunk ID

This separation allows the application to manage transactional data and semantic search efficiently.

14. RAG Pipeline

The RAG pipeline in this project works as follows:

1. Document Loading

- Uploaded PDFs and documents are loaded into the system.

2. Text Chunking

- The document text is divided into smaller chunks for better retrieval.

3. Embedding Generation

- Each chunk is converted into a vector using embedding models.

4. Vector Storage

- The embeddings are stored in ChromaDB with metadata.

5. Similarity Search

- When the user asks a question, the system converts the question into an embedding and searches for similar document chunks.

6. Prompt Construction

- The retrieved chunks are combined with the user question into a prompt.

7. LLM Response

- The language model generates a grounded answer using the retrieved context.

15. Frontend Design

The frontend includes several user-facing pages:

- Login Page

- Dashboard

- Upload Page

- Chat Interface

- Claim Analysis Page

- Document History View

The interface is designed to make document upload, policy search, and claim assistance simple and accessible.

16. Backend API

The backend exposes several API endpoints such as:

```text

POST /login

POST /upload-policy

POST /upload-report

POST /upload-bill

POST /chat

POST /claim

GET /history

```

These endpoints support authentication, document ingestion, retrieval, and claim-related operations.

17. Project Implementation

The project was implemented by combining multiple modules:

- Authentication: user login and access control

- File upload: support for policies and reports

- Text extraction: processing uploaded documents

- Embedding creation: converting text into vector form

- ChromaDB integration: storing and retrieving semantic information

- Retrieval: finding relevant chunks for a query

- LLM response generation: answering questions grounded in the retrieved context

18. Testing

The project can be tested using:

- Unit testing for core logic
- API testing for backend endpoints
- Integration testing for frontend and backend interaction
- Sample queries such as policy coverage questions and claim eligibility questions
- Comparison of expected and actual results

19. Results

The system demonstrates:

- Login and authentication flow
- Dashboard with project metrics
- Upload and indexing of policy documents
- Chat-based insurance assistance
- Claim analysis support
- Retrieved context and grounded responses

Sample screenshots can be included in the final report as follows:

- Login Screen
- Dashboard Screen
- Upload Page
- Chatbot Interface
- Claim Analysis Page
- Retrieved Context View
- Final Response View

20. Advantages

The project offers several advantages:

- Faster policy understanding
- Accurate document retrieval
- Reduced manual effort
- Better customer support
- AI-assisted claim analysis

21. Limitations

The project also has some limitations:

- Performance depends on uploaded document quality
- Retrieval quality depends on embedding quality
- The system does not replace human decision-making for final claim approval

22. Future Enhancements

Possible future enhancements include:

- Multi-language support
- Voice-based assistant
- Image-based document understanding
- Hybrid search
- Multi-agent workflow
- Integration with insurance provider systems

23. Conclusion

This project demonstrates how Retrieval-Augmented Generation can be used to build an intelligent insurance application. By combining document upload, semantic search, and AI-based response generation, the system helps users understand policies and supports claim-related decision-making in a more efficient and reliable way.

24. References

- LangChain Documentation

- ChromaDB Documentation
- FastAPI Documentation
- React Documentation
- Gemini Embedding Documentation
- Research papers and documentation on Retrieval-Augmented Generation